APPLIED AGENTIC AI (AAA) · INTERVIEW PREP

RAG, GraphRAG & Hybrid RAG

An interview preparation FAQ — 52 questions with concise, interview-ready answers

Traditional RAG | Graph RAG | Hybrid RAG

Fundamentals · pipeline & components · GraphRAG · hybrid search & routing
advanced variants · evaluation & production · rapid-fire round

Updated June 2026

Applied Agentic AI (AAA) learning community

How to use this guide

This FAQ collects the questions most commonly asked in interviews for AI / ML / LLM-engineer roles on retrieval systems, with answers written the way a strong candidate would actually respond. Questions are tagged Core Intermediate Advanced so you can pace your prep.

Read this first — "Hybrid RAG" is ambiguous. Interviewers use it two ways: **(1)** hybrid *search* — fusing dense + sparse retrieval; and **(2)** hybrid *architecture* — routing each query between engines such as vector RAG and GraphRAG. A great answer names both, then asks which they mean. Both are covered in Section 4.

1 RAG Fundamentals

1 What is Retrieval-Augmented Generation (RAG), and why does it matter? Core

RAG augments a language model with an external knowledge lookup. Before generating, the system retrieves relevant passages from a knowledge source and places them in the prompt, so the answer is grounded in real, current, or private data rather than only the model's frozen training weights. It matters because it makes LLM outputs more accurate, up to date, and verifiable without retraining the model, and it sharply reduces hallucination by giving the model evidence to anchor to.

2 How is RAG different from a plain LLM, and when would you fine-tune instead? Core

A plain LLM answers only from its parametric memory — fixed at training time, with no access to your private or post-cutoff data. RAG adds a retrieval step so knowledge can change without touching the model. **Fine-tuning** teaches the model new *behaviour, style, or format*; **RAG** supplies new *facts*. If the knowledge changes often or must be auditable, prefer RAG; if you need a consistent skill or tone, fine-tune; many production systems do both.

3 What are parametric and non-parametric memory in this context? Core

Parametric memory is the knowledge baked into the model's weights during training — fast but static and hard to update. Non-parametric memory is the external store (documents, vectors, a graph) that RAG retrieves from at query time — easy to update, inspect, and cite. RAG's value is combining the LLM's parametric fluency with a fresh non-parametric knowledge source.

4 What are the two core components of a RAG system, and how are they coupled?

Core

The **retriever** finds relevant context for a query, and the **generator** (the LLM) writes the answer conditioned on that context. They are coupled through the prompt: the retriever decides what the generator can see. Weak retrieval caps the best possible answer no matter how strong the LLM is, and a weak generator can still waste good retrieval — so both must be evaluated separately.

5 Walk through the stages of a RAG pipeline.

Core

Offline (indexing): load documents → clean/normalize → chunk → embed → store in a vector index (often with metadata). **Online (query time):** embed the query → retrieve top-k (optionally re-rank) → assemble the prompt with the retrieved context → generate → optionally add citations and post-checks. The offline part builds the knowledge base; the online part answers each question.

6 How does RAG reduce hallucinations, and does it eliminate them?

Intermediate

By giving the model retrieved evidence to summarize or quote, RAG shifts the default from guessing to grounding, and a good prompt instructs the model to answer only from context. It does **not** eliminate hallucination: the model can still ignore or misread the passages, or stitch a wrong answer from partially relevant text. You manage this with faithfulness checks, citations, and abstaining when evidence is weak.

7 How does RAG differ from a traditional keyword search engine?

Intermediate

A search engine returns a ranked list of documents and stops; RAG retrieves passages and then synthesizes a direct, natural-language answer from them. Retrieval in modern RAG is usually semantic (embeddings), so it matches meaning rather than only surface keywords, and the generation step can combine evidence across several passages into one coherent response.

8 When is RAG the wrong tool?

Intermediate

When the task needs reasoning or skill rather than facts (e.g. math, code style, tone) — fine-tuning or better prompting helps more. When the whole corpus fits comfortably in context, long-context prompting can be simpler. And for purely creative tasks, retrieval can constrain rather than help. RAG shines when answers must be grounded in a large, changing, or private knowledge base.

2 Retrieval Pipeline & Components

9 Why do we chunk documents, and what makes chunking hard? Core

Chunking splits long documents into passages small enough to embed precisely and fit the context window, while keeping each chunk topically coherent. Too large and retrieval gets imprecise and pollutes the prompt; too small and you fracture meaning across chunks and lose context. Good practice is semantic or structure-aware splitting with some overlap, and tuning size/overlap to your documents.

10 What are embeddings, and what is the difference between dense and sparse retrieval? Core

An embedding is a numeric vector where semantic similarity becomes geometric closeness, so related text lands nearby even without shared words. **Dense retrieval** uses neural embeddings to match meaning; **sparse retrieval** (BM25, TF-IDF) matches exact terms via weighted token overlap. Dense captures paraphrase and intent; sparse nails names, codes, and rare keywords.

11 When does BM25 beat dense retrieval, and vice versa? Intermediate

BM25 wins on exact-match needs — identifiers, product codes, names, quotes, and out-of-vocabulary terms the embedding model never saw. Dense wins on intent-heavy or paraphrased queries where wording differs from the source. Because they fail in different ways, combining them (hybrid search) usually beats either alone.

12 How does a vector database find nearest neighbours efficiently? Intermediate

Exact search over millions of vectors is too slow, so vector DBs use Approximate Nearest Neighbour (ANN) indexes such as HNSW (a navigable small-world graph) or IVF/PQ (inverted lists with quantization). These trade a little recall for large speed and memory gains. Similarity is computed with cosine, dot product, or L2 distance — cosine/dot are most common for normalized text embeddings.

13 How do you choose top-k, and what are the trade-offs? Intermediate

Top-k is how many passages you retrieve. Too small risks missing the passage that holds the answer (low recall); too large pollutes the prompt with noise, raises cost and latency, and can trigger 'lost in the middle' where the model overlooks mid-prompt evidence. A common pattern is retrieve a larger k, then re-rank and keep a small, high-precision set.

14 What is re-ranking, and why use a cross-encoder?

Intermediate

Re-ranking reorders the initial candidate set with a stronger relevance model before generation. A **bi-encoder** (used for first-stage retrieval) embeds query and document separately for speed; a **cross-encoder** reads the query and document together, giving far more accurate relevance at higher cost — ideal for scoring a small candidate set. ColBERT's late interaction is a middle ground that keeps token-level detail more cheaply.

15 What query-transformation techniques improve retrieval?

Advanced

Query rewriting cleans or expands the question; **multi-query** issues several paraphrases and unions the results; **decomposition** splits a complex question into sub-questions; and **HyDE** generates a hypothetical answer and retrieves against it. These help when the user's wording doesn't match how the answer is written in the corpus.

16 Why does prompt construction matter, and what is 'lost in the middle'?

Intermediate

The generator only uses what you put in the prompt and how you frame it, so ordering, de-duplication, and clear instructions ("answer only from context; say if unknown") strongly affect quality. 'Lost in the middle' is the tendency of LLMs to attend best to the start and end of a long context and overlook material in the middle — so put the strongest evidence near the edges and keep context tight.

17 What is metadata filtering and why is it important?

Intermediate

Metadata filtering restricts retrieval by structured attributes — date, source, department, language, document version — before or alongside similarity search. It improves precision, enforces freshness (e.g. latest policy version), and is essential for access control and multi-tenant data boundaries. A classic production bug is a schema change silently breaking filters so the best documents get excluded.

18 What problem does Contextual Retrieval solve?

Advanced

Plain chunks often lose the context they came from (a chunk may say "the rate rose 3%" without saying which rate or year). Contextual Retrieval prepends a short, LLM-generated context blurb to each chunk before embedding/indexing, so isolated chunks stay interpretable and retrievable. It noticeably cuts retrieval-failure rates, especially when combined with hybrid search and re-ranking.

3 GraphRAG

19 What is GraphRAG and how does it differ from vector RAG? Core

GraphRAG indexes knowledge as a **knowledge graph** — entities as nodes, relationships as edges — instead of a flat list of vector chunks. Retrieval becomes graph traversal over connected facts rather than similarity over independent passages. This captures relationships explicitly, so the system can reason across documents and answer corpus-wide questions that flat retrieval cannot.

20 How is the graph built in Microsoft's GraphRAG pipeline? Intermediate

An LLM extracts entities and relationships from each chunk to build a knowledge graph. Densely connected nodes are grouped into **communities** (using hierarchical clustering such as the Leiden algorithm), and the LLM pre-generates a **summary for each community**. This indexing is the expensive, offline step — one reason GraphRAG costs more than embedding.

21 Explain local search vs global search in GraphRAG. Intermediate

Local search answers entity-centric questions: find the entities in the query, walk their neighbourhood, and gather connected facts and chunks. **Global search** answers whole-corpus, thematic questions (e.g. "what are the main themes?"): it maps over the community summaries to produce partial answers, rates and filters them, then reduces them into one global answer.

22 Why does GraphRAG beat naive RAG on global and multi-hop questions? Core

Global questions are really query-focused summarization over the entire corpus — top-k similarity only ever sees a few chunks, so it structurally cannot answer them; community summaries can. Multi-hop questions need facts that live in different, not-mutually-similar chunks (A→B→C); the graph stores those links explicitly, so traversal assembles the full chain that vector search fragments.

23 What are nodes, edges, and communities? Core

Nodes are entities (people, organizations, products, concepts). **Edges** are typed relationships between them (founded, acquired, builds, located-in). **Communities** are clusters of densely connected nodes; their LLM-written summaries let the system reason about themes at different levels of granularity.

24 What are the cost and latency trade-offs of GraphRAG? Intermediate

Indexing is much heavier than vector RAG because it runs many LLM calls for entity/relationship extraction and community summarization, and updates mean re-running parts of that pipeline. Query latency is usually higher too, especially global search. You pay this premium to gain multi-hop reasoning, global summarization, and traceability — so it's justified mainly for hard, interconnected corpora.

25 When should you choose GraphRAG, and what are typical use cases?

Core

Choose it when data is highly interconnected, questions are exploratory or multi-hop, and explainability matters. Typical domains: healthcare and biomedical Q&A, legal and case-law research, enterprise knowledge management, compliance and audit reasoning, and fraud or investigation graphs.

26 How do graph databases and Cypher fit in?

Advanced

Production GraphRAG often stores the knowledge graph in a graph database such as Neo4j, and retrieval can translate a natural-language question into a **Cypher** query (Neo4j's graph query language) that traverses the graph and returns a subgraph as context. Tools like Neo4j's LLM graph builder help construct the graph from unstructured text.

27 What are the main challenges with GraphRAG?

Advanced

Extraction quality (noisy or inconsistent entities/relations degrade everything downstream), entity resolution/canonicalization, scaling to graphs with millions of nodes, and keeping the graph fresh as source data changes. Indexing cost and slower iteration are practical hurdles, which is why many teams use it only for the hard subset of queries.

4 Hybrid RAG

28 "Hybrid RAG" is ambiguous — what are the two meanings?

Core

Clarify which one the interviewer means. **(1) Hybrid search / retrieval:** combine *dense* (semantic) and *sparse* (BM25/keyword) retrieval and fuse the results — the most common meaning. **(2) Hybrid architecture:** route each query between different engines, e.g. vector RAG vs GraphRAG, picking the right one per query. Strong answers name both and then dive into the relevant one.

29 What is hybrid search and why does it outperform either method alone?

Core

Hybrid search runs dense and sparse retrieval in parallel and merges their results. Dense captures semantic intent and paraphrase; sparse captures exact terms, names, and rare keywords. Because they fail on different queries, their union raises recall and their fusion raises precision, consistently beating either retriever by itself across benchmarks.

30 How do you fuse dense and sparse results? Explain RRF and weighted scoring.

Intermediate

Two common methods. **Reciprocal Rank Fusion (RRF):** score each document by summing 1/(k + rank) across the two ranked lists (k≈60), which needs no score calibration and is robust. **Weighted score fusion:** combine normalized similarity scores as λ·dense + (1−λ)·sparse, tuning λ to your data. RRF is popular because it avoids comparing incompatible score scales.

31 How do you tune the dense/sparse balance?

Intermediate

Treat λ (or per-channel weights) as a hyperparameter and tune it on a labeled query set using retrieval metrics (recall@k, MRR, nDCG). Keyword-heavy domains (legal, code, finance with tickers) lean sparse; conversational or paraphrase-heavy domains lean dense. Validate per query type, since one global weight rarely suits every query.

32 Describe the architecture-level hybrid: routing between vector RAG and GraphRAG.

Intermediate

Put a lightweight **router** in front that classifies each query and dispatches it: simple factual lookups go to fast/cheap vector RAG, multi-hop or thematic questions go to GraphRAG, and ambiguous ones can run both and merge. This pays the graph's cost only when a query needs it while keeping vector RAG's speed for the easy majority.

33 How does the router decide, and how would you start simple?

Intermediate

Start with cheap heuristics (query length, keywords, presence of multiple entities, "how/why/compare" patterns), then graduate to an LLM classifier or a small trained model as you learn real traffic. Frameworks like LangGraph or LlamaIndex routers formalize this. Log routing decisions so you can audit and improve them.

34 How would you combine GraphRAG, vector search, and re-ranking in one pipeline?

Advanced

Retrieve candidates from both a vector store and the graph (entities/subgraphs), union them, then apply a cross-encoder re-ranker to produce one high-precision context set for generation. An agent or router can decide per query whether to lean on graph traversal, hybrid vector search, or both — adapting the strategy to the question.

35 What are the downsides of hybrid systems?

Advanced

More moving parts: two or more indexes to build and keep in sync, fusion/routing logic to tune, higher latency and cost, and harder debugging and evaluation. The art is adding hybridization only where it measurably improves answers, and keeping the simplest path (plain vector RAG) for queries that don't need more.

5 RAG vs GraphRAG vs Hybrid

36 Compare RAG, GraphRAG, and Hybrid side by side.

Core

Factor	Traditional RAG	Graph RAG	Hybrid
Best question type	Fact-based, local	Exploratory, multi-hop	Mixed workloads
Accuracy on complex Qs	Medium	Very high	High–Very high
Speed / latency	Fast	Slower	Routed
Explainability	Lower	Higher (traceable paths)	Higher
Update / freshness	Fast (re-embed)	Slower (re-index)	Mixed
Relative cost	Low	Higher	Medium–High
Typical use	FAQ, doc search	Healthcare, legal, KM	Enterprise ask-your-docs

37 Give a quick decision framework for choosing among them.

Core

Fact-based and answerable from one place → **Traditional RAG**. Spans many documents or needs multi-hop reasoning → **Graph RAG**. Mixed traffic or unsure per query → **Hybrid + routing**. Tight latency/budget with simple data → stay with **Traditional RAG**. In practice teams start with RAG, add Graph for the hard ~20%, then formalize the choice as a router.

38 How do these approaches differ in explainability?

Intermediate

Vector RAG can cite which chunks it used, but the link between them is implicit. GraphRAG can show the actual reasoning path — the chain of entities and relationships, each traceable to a source document — which is valuable in regulated domains. Hybrid inherits the graph's traceability when it routes to the graph path.

39 How do freshness and update costs compare?

Intermediate

Vector RAG updates are cheap: re-embed and upsert the changed chunks. GraphRAG updates are costlier because new data may require re-extraction and re-clustering of communities (some systems support incremental updates to soften this). Hybrid sits in between, depending on which index a change touches.

6 Advanced RAG Variants

40 What is Self-RAG?

Advanced

Self-RAG trains/prompts the model to decide *when* to retrieve and to critique its own output using reflection signals — judging whether retrieval is needed, whether passages are relevant, and whether the answer is supported. It retrieves on demand and self-corrects, improving faithfulness over always-retrieve pipelines.

41 What is Corrective RAG (CRAG)?

Advanced

CRAG adds a lightweight evaluator that grades retrieved documents and triggers corrective action when they're weak — for example, refining the query or falling back to web search — before generation. It guards against confidently answering from poor evidence.

42 What is Adaptive RAG?

Advanced

Adaptive RAG decides the retrieval strategy (or whether to retrieve at all) based on query complexity or model confidence — sending easy questions down a cheap path and hard ones down a richer path. It's essentially the routing idea applied to retrieval depth, balancing cost and quality.

43 What is Agentic RAG?

Advanced

Agentic RAG wraps retrieval in an agent loop: an LLM plans, chooses tools, retrieves, grades results, rewrites queries, and iterates until the answer is grounded — often modeled as a graph of nodes (retrieve, grade, rewrite, web-search, generate, reflect) in LangGraph/CrewAI/AutoGen. It delivers stronger multi-hop, self-correcting answers at the cost of latency, tokens, complexity, and non-determinism.

44 What is RAPTOR?

Advanced

RAPTOR (Recursive Abstractive Processing for Tree-Organized Retrieval) builds a hierarchical tree of summaries by recursively clustering and summarizing chunks. Retrieval can then pull from different abstraction levels, supporting both broad and specific questions over the same corpus — useful for reasoning across whole documents.

45 What is RAFT (Retrieval-Augmented Fine-Tuning)?

Advanced

RAFT fine-tunes a model to use retrieved context well: training examples include relevant passages plus distractor passages, teaching the model to cite the right evidence and ignore noise. It blends RAG's fresh knowledge with fine-tuning's reliability for a specific domain.

7 Evaluation & Production

46 How do you evaluate a RAG system? Name key metrics.

Core

Evaluate retrieval and generation separately, on a golden Q&A set. Retrieval: recall@k, precision, MRR, nDCG. Generation/end-to-end: **faithfulness** (is the answer supported by the context?), **answer relevancy**, and **context relevancy/recall** — the metrics popularized by frameworks like **RAGAS**. Add LLM-as-judge graders and run them as regression gates on every change.

47 A RAG answer is wrong — how do you tell retrieval vs generation failure?

Intermediate

Retrieval failure: the top-k passages don't contain the answer (off-topic, low similarity, missing entities) — the generator never had a chance. **Generation failure:** the passages do contain the answer but the model ignores, misreads, or over-claims beyond it. You diagnose by checking the retrieved context first, then the answer's faithfulness against it.

48 What are common production failure modes?

Intermediate

Stale indexes after data changes, bad chunking, missing/broken metadata filters, embedding drift after a model update, oversized top-k polluting the prompt, re-ranker latency spikes, prompt injection via documents, and 'citation laundering' where citations exist but don't actually support the claim. Most 'LLM problems' turn out to be retrieval problems.

49 How would you reduce latency in a live RAG system?

Intermediate

Use ANN indexes for fast retrieval, cap and re-rank top-k instead of stuffing context, cache embeddings and frequent queries, pre-compute where possible, and route simple queries to cheaper paths. Reserve heavy steps (cross-encoder re-ranking, global graph search, agentic loops) for queries that need them.

50 What security risks are specific to RAG, and how do you mitigate them?

Advanced

Documents are untrusted input, so a poisoned document can carry a **prompt injection** that hijacks the model. Mitigations: treat retrieved content as data not instructions, sanitize/validate sources, isolate tool actions behind approvals, and enforce access control so retrieval respects per-user/tenant data boundaries. Also guard against leaking one tenant's data via a shared index.

51 How do you handle multi-turn conversations in RAG?

Intermediate

Carry relevant context across turns by rewriting the new question with information from prior turns (query refinement), so the retriever gets a self-contained, context-rich query. Optionally maintain a running summary or memory of the conversation, and avoid dumping the entire history into every retrieval call.

52 **How do you treat the RAG stack as production software?** Advanced

Version documents, embeddings, and indexes; test the ingestion pipeline; roll out from dev → canary → prod; keep chunk IDs backward-compatible to allow rollbacks; and log exactly which data and model versions produced each answer so every response is auditable and reproducible.

8 · Rapid-fire round

Sparse vs dense in one line?	Sparse = exact keyword match (BM25); dense = semantic match via embeddings.
What is RRF's typical constant?	k ≈ 60 in 1/(k + rank).
Cosine vs dot vs L2?	All measure vector closeness; cosine/dot are standard for normalized text embeddings.
Bi-encoder vs cross-encoder?	Bi-encoder = fast first-stage retrieval; cross-encoder = accurate re-ranking.
What does HyDE do?	Generates a hypothetical answer and retrieves against it.
Algorithm for graph communities?	Hierarchical clustering, e.g. Leiden.
Graph query language for Neo4j?	Cypher.
Two GraphRAG query modes?	Local (entity neighbourhood) and global (community summaries).
Core RAGAS metrics?	Faithfulness, answer relevancy, context relevancy/recall.
What is 'lost in the middle'?	LLMs under-attend to evidence in the middle of long contexts.
One-line fix for confident-but-ungrounded answers?	Instruct to answer only from context and abstain on weak evidence; check faithfulness.
RAG or fine-tune for new facts?	RAG for facts; fine-tune for behaviour/skill/format.

References & further reading

- DataCamp — Top 30 RAG Interview Questions (2026)
  https://www.datacamp.com/blog/rag-interview-questions
- Analytics Vidhya — 40 RAG Interview Questions & Answers
  https://www.analyticsvidhya.com/blog/2026/02/rag-interview-questions-and-answers/
- ProjectPro — Top 30 RAG Interview Questions
  https://www.projectpro.io/article/rag-interview-questions-and-answers/1065
- Cloud Soft Solutions — 45 RAG Interview Questions (2026)
  https://cloudsoftsol.com/interview-questions/rag-interview-questions-2026/

- S. Kumar — Interview Questions: GraphRAG vs Traditional RAG
  https://skphd.medium.com/interview-questions-on-graphrag-vs-traditional-rag-6e9492249f9e
- Edge et al., 2024 — From Local to Global: A Graph RAG Approach (arXiv:2404.16130)
  https://arxiv.org/abs/2404.16130
- Guo et al., 2024 — LightRAG (arXiv:2410.05779)
  https://arxiv.org/abs/2410.05779
- microsoft/graphrag
  https://github.com/microsoft/graphrag
- HKUDS/LightRAG
  https://github.com/HKUDS/LightRAG
- NirDiamant/RAG_Techniques
  https://github.com/NirDiamant/RAG_Techniques
- Anthropic — Introducing Contextual Retrieval
  https://www.anthropic.com/news/contextual-retrieval

Compiled for the Applied Agentic AI (AAA) learning community. Answers are original summaries synthesized from the sources above and current practice as of June 2026; figures and star counts are approximate. Reuse and adapt freely.